



考点1 内部排序思想

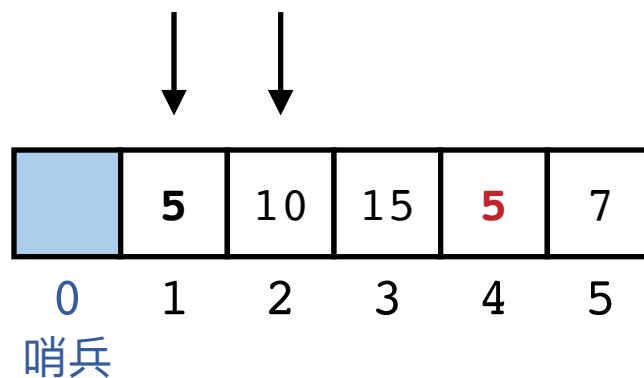
①直接插入排序

①A[0]作为哨兵位

②未排序序列的首个元素A[i] ($i \geq 2$) 会**从后往前**和已有序序列的元素逐个对比

①如果A[i] $\geq$ 已有序序列中的某个元素，不做操作

②如果A[i] $<$ 已有序序列中的某个元素，待插入元素A[i]被复制到哨兵位，将A[i]之前大于哨兵的元素后移一个位置，继续进行对比，直到确定A[i]的插入位置后，将哨兵位的元素复制到腾出的空位上





考点1 内部排序思想

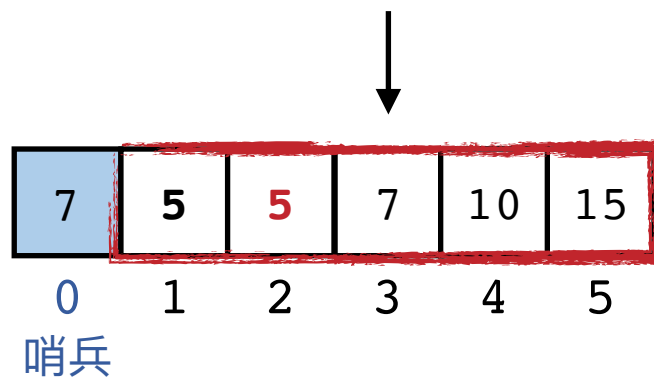
①直接插入排序

①A[0]作为哨兵位

②未排序序列的首个元素A[i] ($i \geq 2$) 会**从后往前**和已有序序列的元素逐个对比

①如果 $A[i] \geq$ 已有序序列中的某个元素，不做操作

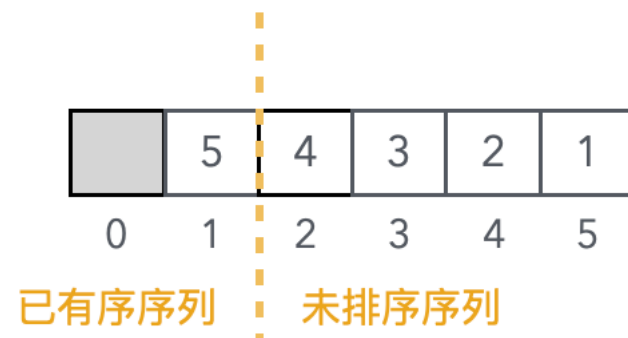
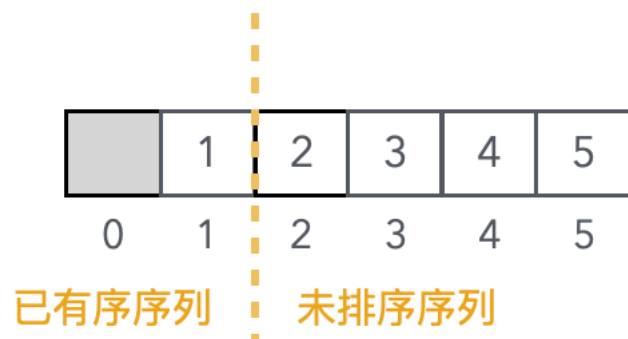
②如果 $A[i] <$ 已有序序列中的某个元素，待插入元素A[i]被复制到哨兵位，将A[i]之前大于哨兵的元素后移一个位置，继续进行对比，直到确定A[i]的插入位置后，将哨兵位的元素复制到腾出的空位上





考点1 内部排序思想

①直接插入排序



最好情况

- ①未排序序列的每个元素只需要和前面的一个元素进行比较后就可插入已有序序列
- ②总比较次数 = $1+1+\dots+1 = n-1$
- ③总移动次数 = 0

最坏情况

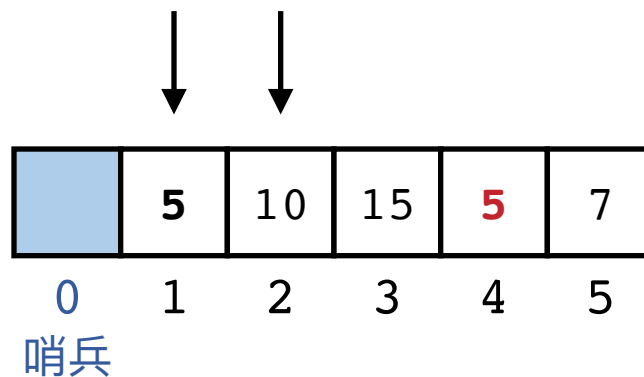
- ①未排序序列的每个元素需要和已有序序列的每个元素进行比较后才能插入已有序序列
- ②总比较次数 = $1+2+3+\dots+n-1 = \frac{n \times (n-1)}{2}$
- ③总移动次数 = $3+4+\dots+n+1 = \frac{(4+n) \times (n-1)}{2}$



考点1 内部排序思想

②折半插入排序

- ①折半插入排序可以优化直接插入排序的比较次数，但没有改变移动次数，所以折半插入排序的平均时间复杂度仍是 $O(n^2)$
- ②折半插入排序中，每一趟的比较次数**最多**为 $\lceil \log_2(i+1) \rceil$ （ i 位已有序序列的元素个数）
- ③折半插入排序只适用顺序表

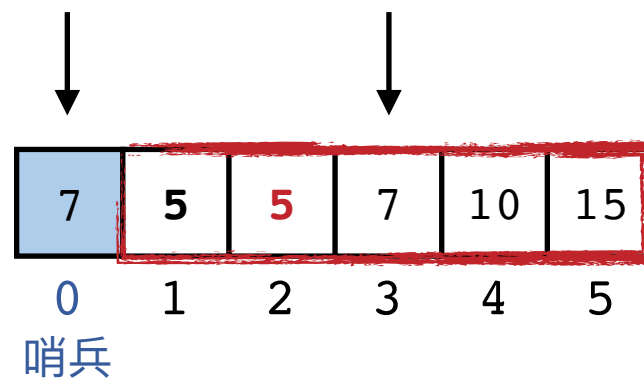




考点1 内部排序思想

②折半插入排序

- ①折半插入排序可以优化直接插入排序的比较次数，但没有改变移动次数，所以折半插入排序的平均时间复杂度仍是 $O(n^2)$
- ②折半插入排序中，每一趟的比较次数**最多**为 $\lceil \log_2(i+1) \rceil$ （ i 位已有序序列的元素个数）
- ③折半插入排序只适用顺序表





考点1 内部排序思想

②折半插入排序

折半插入排序所需要的关键字比较次数与待排序序列的初始排列无关，仅依赖于记录的个数。不论初始序列情况如何，在插入第 i 个记录时，需要经过 $\lfloor \log_2 i \rfloor + 1$ 次比较，才能确定它应插入的位置。所以当记录的初始排列为正序或接近正序时，直接插入排序比折半插入排序执行的关键字比较次数要少。

最佳情况：当目标值位于数组的中间，在这种情况下，正好每次进行1次比较

平均情况：数量级固定为 $\log_2 i$ 次，因为每次比较时，折半查找都会将数组切成两半，但这是一个近似值，确切数字会根据初始序列的情况略有不同

最坏情况：当目标值不在数组中，在这种情况下，正好进行 $\lceil \log_2(i + 1) \rceil$ 的比较

大题考频：3/16，分值：10~15

③希尔排序

- ①将待排序表相隔几个元素分割成几个子表，第一轮 $dk=n/2$ ，第二轮 $dk=dk/2$ ，以此类推
- ②对每个子表分别进行直接插入排序
- ③整个表中的元素已基本有序时，再对全体记录进行一次直接插入排序

	5	10	15	5	7
0	1	2	3	4	5

哨兵

考点1 内部排序思想

③希尔排序

- ①将待排序表相隔几个元素分割成几个子表，第一轮 $dk=n/2$ ，第二轮 $dk=dk/2$ ，以此类推
- ②对每个子表分别进行直接插入排序
- ③整个表中的元素已基本有序时，再对全体记录进行一次直接插入排序

$dk=2$

$dk=1$

5	5	5	7	10	15
0	1	2	3	4	5

哨兵

希尔排序是不稳定的排序算法，这里的相对位置没变只是巧合



考点1 内部排序思想

③希尔排序

- ①将待排序表相隔几个元素分割成几个子表，第一轮 $dk=n/2$ ，第二轮 $dk=dk/2$ ，以此类推
- ②对每个子表分别进行直接插入排序
- ③整个表中的元素已基本有序时，再对全体记录进行一次直接插入排序

希尔排序是不稳定的排序算法，这里的相对位置没变只是巧合
这种情况就能体现出不稳定性

$dk=2$

	1	7	5	5	15
--	---	---	---	---	----

0 1 2 3 4 5

哨兵



考点1 内部排序思想

③希尔排序

- ①将待排序表相隔几个元素分割成几个子表，第一轮 $dk=n/2$ ，第二轮 $dk=dk/2$ ，以此类推
- ②对每个子表分别进行直接插入排序
- ③整个表中的元素已基本有序时，再对全体记录进行一次直接插入排序

希尔排序是不稳定的排序算法，这里的相对位置没变只是巧合
这种情况就能体现出不稳定性

$dk=2$

$dk=1$

5	1	5	5	7	15
0	1	2	3	4	5

哨兵



考点1 内部排序思想

④冒泡排序

- ①不断地让相邻的两个元素进行比较，每一趟确定一个最小（最大）值的最终位置
- ②可以从左往右，也可以从右往左
- ③当上一趟没有发生任何交换时，可以直接停止冒泡排序

某一轮未发生交换即为排序完成

	5	5	7	10	15
0	1	2	3	4	5

哨兵

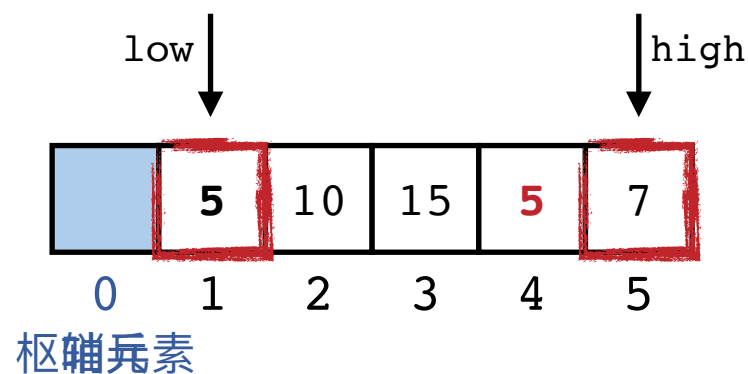


考点1 内部排序思想

⑤快速排序

- ①选择一个元素作为枢轴元素（通常是表中第一个元素）
- ②表的左右侧设立low和high指针，初始时low=1，high=L.length
- ③high指针往左走，碰到比枢纽元素小的就把元素放到low位置，然后操作low指针
low指针往右走，碰到比枢纽元素大的就把元素放到high位置，然后操作high指针

快排用不上哨兵，但是需要临时存储枢轴元素，这里为展示方便就让枢轴元素存在哨兵位置





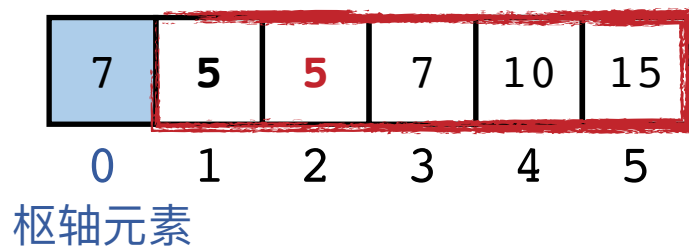
考点1 内部排序思想

⑤快速排序

- ①选择一个元素作为枢轴元素（通常是表中第一个元素）
- ②表的左右侧设立low和high指针，初始时low=1，high=L.length
- ③high指针往左走，碰到比枢纽元素小的就把元素放到low位置，然后操作low指针
low指针往右走，碰到比枢纽元素大的就把元素放到high位置，然后操作high指针

快排用不上哨兵，但是需要临时存储枢轴元素，这里为展示方便就让枢轴元素存在哨兵位置

这里的稳定也是巧合，实际快排是不稳定算法





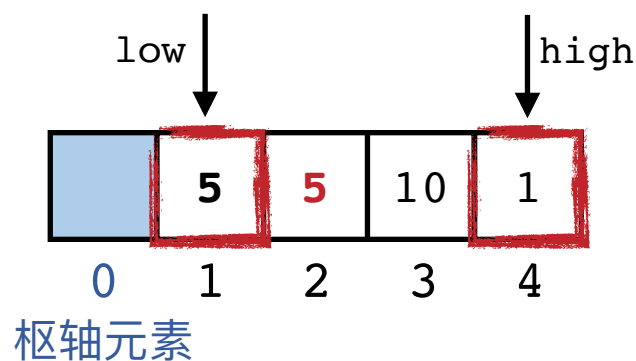
考点1 内部排序思想

⑤快速排序

- ①选择一个元素作为枢轴元素（通常是表中第一个元素）
- ②表的左右侧设立low和high指针，初始时 $low=1$ ， $high=L.length$
- ③high指针往左走，碰到比枢纽元素小的就把元素放到low位置，然后操作low指针
low指针往右走，碰到比枢纽元素大的就把元素放到high位置，然后操作high指针

快排用不上哨兵，但是需要临时存储枢轴元素，这里为展示方便就让枢轴元素存在哨兵位置

这里的稳定也是巧合，实际快排是不稳定算法





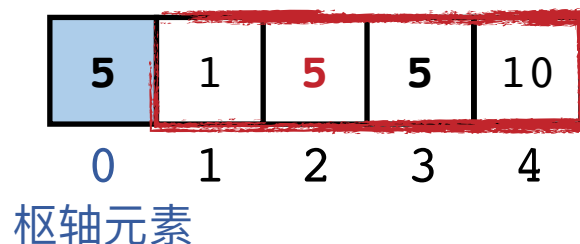
考点1 内部排序思想

⑤快速排序

- ①选择一个元素作为枢轴元素（通常是表中第一个元素）
- ②表的左右侧设立low和high指针，初始时low=1，high=L.length
- ③high指针往左走，碰到比枢纽元素小的就把元素放到low位置，然后操作low指针
low指针往右走，碰到比枢纽元素大的就把元素放到high位置，然后操作high指针

快排用不上哨兵，但是需要临时存储枢轴元素，这里为展示方便就让枢轴元素存在哨兵位置

这里的稳定也是巧合，实际快排是不稳定算法





考点1 内部排序思想

⑤快速排序

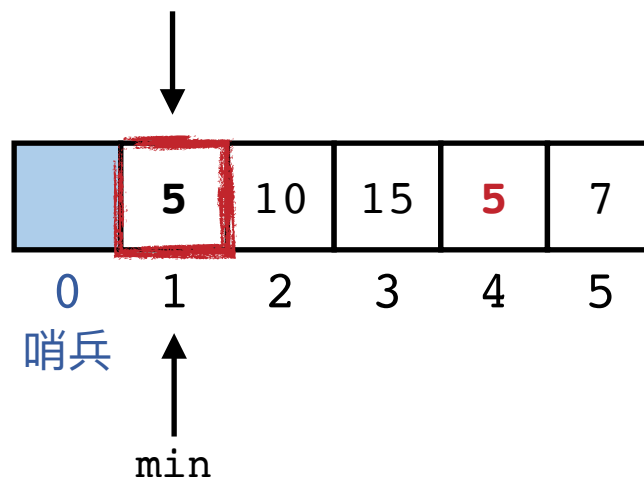
- ①当子表长度为1时，代码实际上会再次对这个元素调用一次快排，但手动模拟的时候我们一般忽略这一次调用
- ②对一个子表调用一次快排**至少**能够确定一个元素的最终位置（6，1，17），pivot一定会确定最终位置，且左边的元素都比它小，右边的元素都比它大
- ③快排的最好情况是每次选择的pivot刚好可以把表分成相对均匀的两份，最坏情况是初始序列已基本有序
- ④快排递归树的树高和递归深度一致，最大递归深度为n，最小递归深度为 $\log_2$
- ⑤快排依赖随机访问的特性，因此适用于顺序存储



考点1 内部排序思想

⑥简单选择排序

- ①每一趟在无序序列中找到最小的元素与无序序列的第一个元素交换，直到整体有序
- ②每一趟设置一个最小值指针，初始时指向无序序列的第一个元素，遍历结束后指向本趟无序序列中的最小值

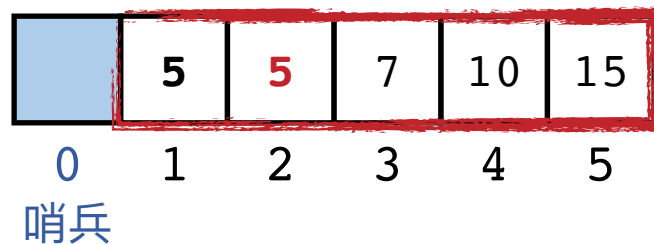




考点1 内部排序思想

⑥简单选择排序

- ①每一趟在无序序列中找到最小的元素与无序序列的第一个元素交换，直到整体有序
- ②每一趟设置一个最小值指针，初始时指向无序序列的第一个元素，遍历结束后指向本趟无序序列中的最小值





考点1 内部排序思想

⑦堆排序（以大根堆为例）

①建堆：

- ①从最后一个分支节点所在子树开始，根节点与较大孩子互换
- ②逐个从后往前让所有子树满足大根堆定义
- ③如果在调整的过程中出现后面的调整影响到了前面调整好的子树，应立刻将被影响的部分恢复好，再继续往前进行调整

②调整堆：

- ①交换堆顶和堆底元素
- ②从上到下把受到影响的部分重新调整为大根堆

③堆的删除：删除堆顶元素，再用堆底元素填补空位，从上到下调整堆

堆的插入：新元素被插入到堆底，插入后需要从下往上调整堆

④左右孩子的值相等时，选择左孩子和父亲进行交换

⑤堆是一个完全二叉树

考点1 内部排序思想

⑦堆排序（以大根堆为例）

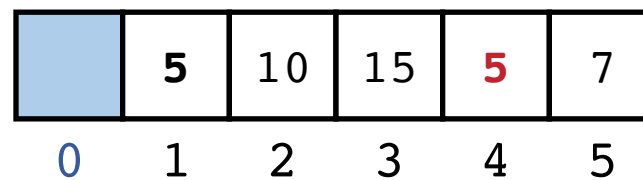
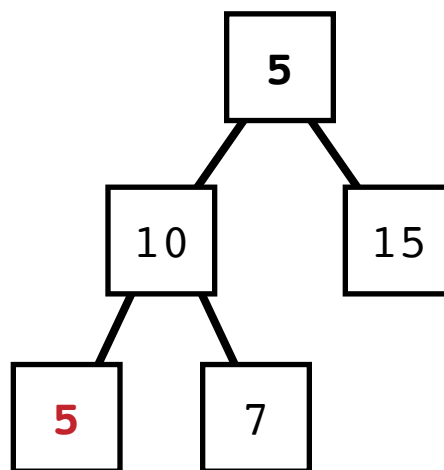
5	10	15	5	7
---	----	----	---	---

	5	10	15	5	7
0	1	2	3	4	5



考点1 内部排序思想

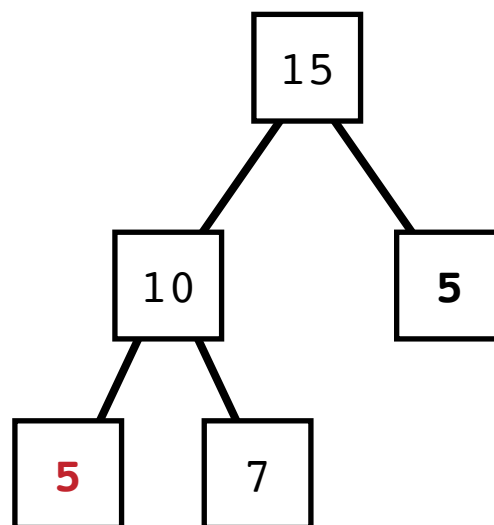
⑦堆排序（以大根堆为例）



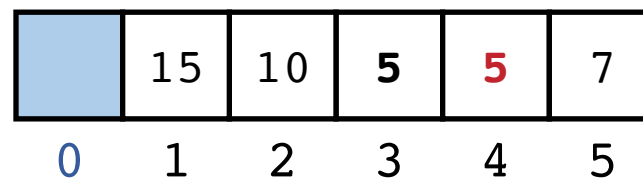


考点1 内部排序思想

⑦堆排序（以大根堆为例）



建堆完成，开始排序



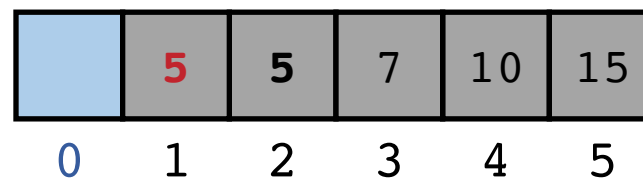
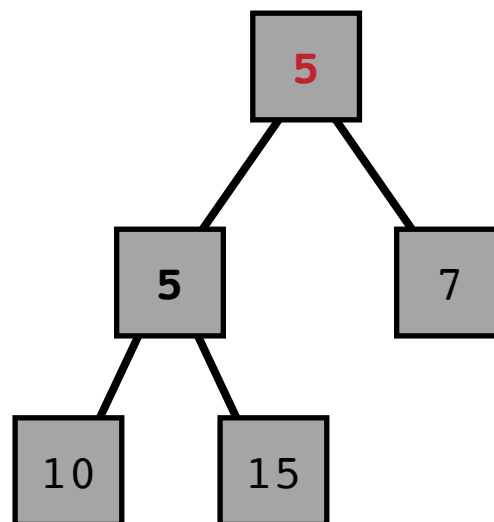


选择题考频：16/16，分值：2

大题考频：3/16，分值：10~15

考点1 内部排序思想

⑦堆排序（以大根堆为例）



考点1 内部排序思想

⑦堆排序（以大根堆为例）

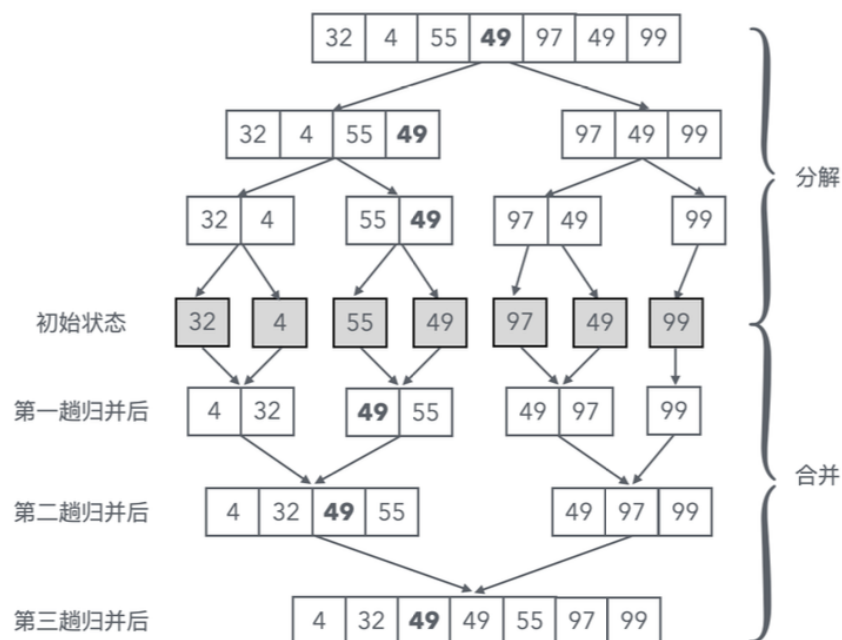
	5	5	7	10	15
0	1	2	3	4	5



考点1 内部排序思想

⑧归并排序（以二路归并排序为例）

- ①假设初始序列含有 n 个记录，则可将其看成 n 个有序的子序列，每个子序列的长度为1，然后两两归并，得到 $\lceil n/2 \rceil$ 个长度为2或1的有序子序列，再两两归并，如此重复，直至得到一个长度为 n 的有序序列为止
- ② N 个元素进行 k 路归并，趟数 = $\lceil \log_k N \rceil$
- ③长度为 m 和 n 的有序表进行二路归并，最多比较 $m+n-1$ 次，最少比较 $\min\{m, n\}$ 次
- ④归并排序的分和合是两个对称的过程，考虑代码时要注意不一定是相邻两个进行归并

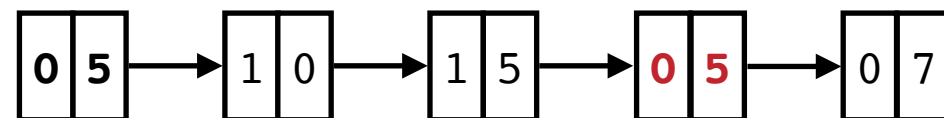




考点1 内部排序思想

⑨基数排序

- ①不比较关键字的大小，而是根据关键字中各位的值，对排序记录进行若干趟分配和收集来实现排序
- ②默认使用LSD低位优先法

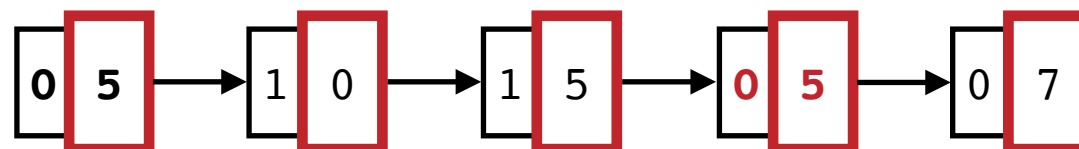




考点1 内部排序思想

⑨基数排序

- ①不比较关键字的大小，而是根据关键字中各位的值，对排序记录进行若干趟分配和收集来实现排序
- ②默认使用LSD低位优先法



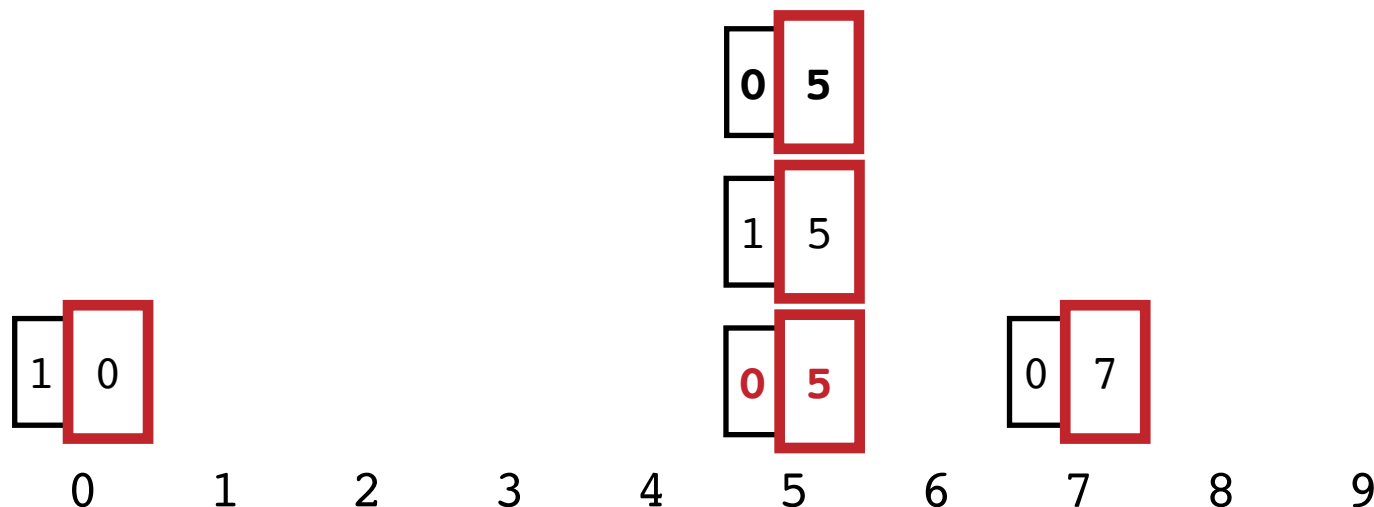
0 1 2 3 4 5 6 7 8 9



考点1 内部排序思想

⑨基数排序

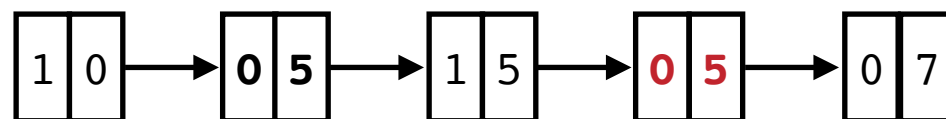
- ①不比较关键字的大小，而是根据关键字中各位的值，对排序记录进行若干趟分配和收集来实现排序
- ②默认使用LSD低位优先法



考点1 内部排序思想

⑨基数排序

- ①不比较关键字的大小，而是根据关键字中各位的值，对排序记录进行若干趟分配和收集来实现排序
- ②默认使用LSD低位优先法

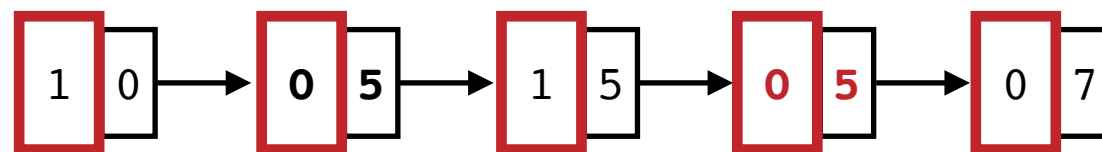


0 1 2 3 4 5 6 7 8 9

考点1 内部排序思想

⑨基数排序

- ①不比较关键字的大小，而是根据关键字中各位的值，对排序记录进行若干趟分配和收集来实现排序
- ②默认使用LSD低位优先法



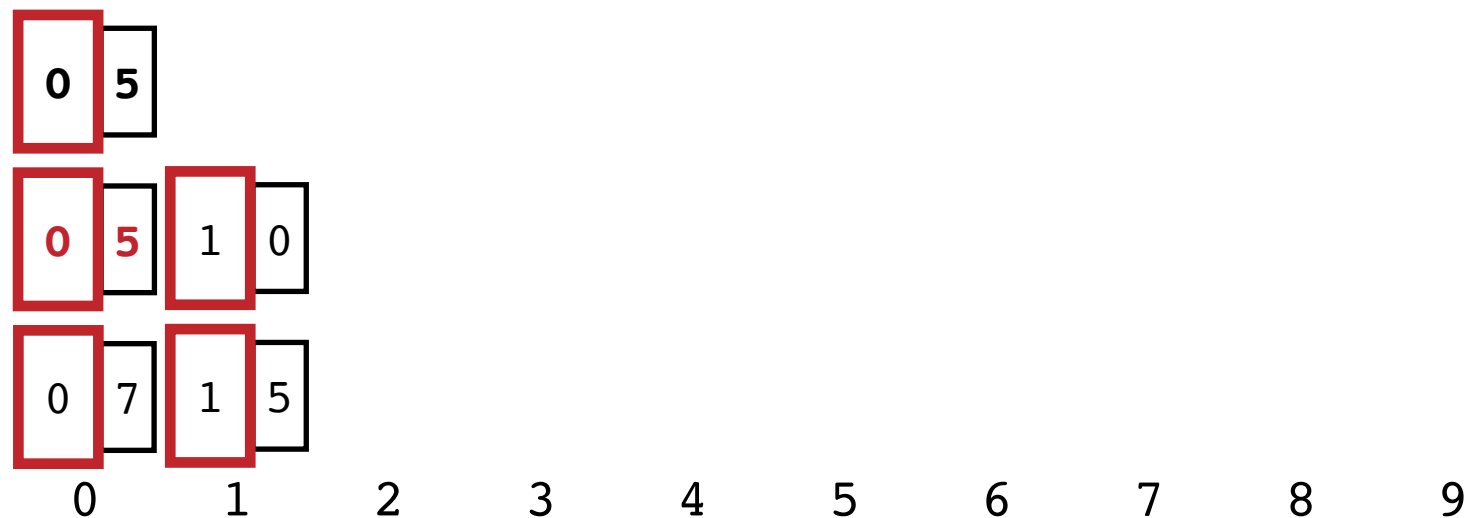
0 1 2 3 4 5 6 7 8 9



考点1 内部排序思想

⑨基数排序

- ①不比较关键字的大小，而是根据关键字中各位的值，对排序记录进行若干趟分配和收集来实现排序
- ②默认使用LSD低位优先法

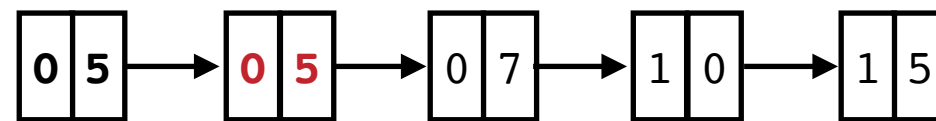




考点1 内部排序思想

⑨基数排序

- ①不比较关键字的大小，而是根据关键字中各位的值，对排序记录进行若干趟分配和收集来实现排序
- ②默认使用LSD低位优先法



考点2 内部排序总结

①稳定性

有两个关键字相同的元素，经过排序后它们的相对顺序仍然不变，则算法是稳定的

算法	稳定性
直接插入排序	
折半插入排序	
希尔排序	不稳定
冒泡排序	
快速排序	不稳定
简单选择排序	不稳定
堆排序	不稳定
归并排序	
基数排序	

口诀：选艾希堆攻速

考点2 内部排序总结

②时空复杂度

算法	时间复杂度			空间复杂度
	最好	最坏	平均	
直接插入排序	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$
冒泡排序	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$
简单选择排序	$O(n^2)$			$O(1)$
快速排序	$O(n\log_2n)$	$O(n^2)$	$O(n\log_2n)$	$O(\log_2n) \sim O(n)$
堆排序	$O(n\log_2n)$			$O(1)$
归并排序	$O(n\log_2n)$			$O(n)$
基数排序	$O(d(n+r))$ d 趟分配， n 个关键字， r 个队列			$O(r)$

时间复杂度与初始序列状态无关的算法：简单选择排序、堆排序、归并排序、基数排序



考点2 内部排序总结

③一趟排序能**确定一个元素最终位置**的算法

直接插入排序
折半插入排序
希尔排序
冒泡排序
简单选择排序
快速排序
堆排序
归并排序
基数排序



考点2 内部排序总结

④比较次数与序列的初始状态无关的算法

直接插入排序
折半插入排序（数量级）
希尔排序
冒泡排序（未优化）
简单选择排序
快速排序
堆排序
归并排序（数量级）
基数排序

考点2 内部排序总结

⑤移动次数与序列的初始状态无关的算法

直接插入排序
折半插入排序
希尔排序
冒泡排序
简单选择排序
快速排序
堆排序
归并排序
基数排序



考点2 内部排序总结

⑥总趟数与序列的初始状态有关的算法

直接插入排序
折半插入排序
希尔排序
冒泡排序（优化后的）
简单选择排序
快速排序
堆排序
归并排序
基数排序



考点2 内部排序总结

【2009】若数据元素序列{11, 12, 13, 7, 8, 9, 23, 4, 5}是采用下列排序算法之一得到的第二趟排序后的结果，则该排序算法只能是

- A. 冒泡排序
- B. 插入排序
- C. 选择排序
- D. 二路归并排序



考点2 内部排序总结

【2009】若数据元素序列{11, 12, 13, 7, 8, 9, 23, 4, 5}是采用下列排序算法之一得到的第二趟排序后的结果，则该排序算法只能是

A. 冒泡排序

B. 插入排序

C. 选择排序

D. 二路归并排序



考点2 内部排序总结

【2017】在内部排序时，若选择了归并排序而未选择插入排序，则可能的理由是

- I. 归并排序的程序代码更短
- II. 归并排序的占用空间更少
- III. 归并排序的运行效率更高

- A. 仅II
- B. 仅III
- C. 仅I、II
- D. 仅I、III



考点2 内部排序总结

【2017】在内部排序时，若选择了归并排序而未选择插入排序，则可能的理由是

- I. 归并排序的程序代码更短
- II. 归并排序的占用空间更少
- III. 归并排序的运行效率更高

A. 仅II

B. 仅III

C. 仅I、II

D. 仅I、III



考点2 内部排序总结

【2022】对数据进行排序时，若采用直接插入排序而不采用快速排序，则可能的原因是

- I. 大部分元素已有序
 - II. 待排序元素数量很少
 - III. 要求空间复杂度为 $O(1)$
 - IV. 要求排序算法是稳定的
- A. 仅I、II
 - B. 仅III、IV
 - C. 仅I、II、IV
 - D. I、II、III、IV



考点2 内部排序总结

【2022】对数据进行排序时，若采用直接插入排序而不采用快速排序，则可能的原因是

- I. 大部分元素已有序
- II. 待排序元素数量很少
- III. 要求空间复杂度为 $O(1)$
- IV. 要求排序算法是稳定的

- A. 仅I、II
- B. 仅III、IV
- C. 仅I、II、IV

D. I、II、III、IV



考点2 内部排序总结

【2012】用希尔排序算法对一个数据序列进行排序时，若第一趟排序结果为9，1，4，13，7，8，20，23，15，用该趟排序采用的增量（间隔）可能是

- A. 2
- B. 3
- C. 4
- D. 5

考点2 内部排序总结

【2012】用希尔排序算法对一个数据序列进行排序时，若第一趟排序结果为9，1，4，13，7，8，20，23，15，用该趟排序采用的增量（间隔）可能是

A. 2

B. 3

C. 4

D. 5



考点2 内部排序总结

【2019】排序过程中，对尚未确定最终位置的所有元素进行一边处理称为一趟。下列序列中，不可能是快速排序第二趟结果是

- A. 5, 2, 16, 12, 28, 60, 32, 72
- B. 2, 16, 5, 28, 12, 60, 32, 72
- C. 2, 12, 16, 5, 28, 32, 72, 60
- D. 5, 2, 12, 28, 16, 32, 72, 60



考点2 内部排序总结

【2019】排序过程中，对尚未确定最终位置的所有元素进行一边处理称为一趟。下列序列中，不可能是快速排序第二趟结果是

A. 5, 2, 16, 12, 28, 60, 32, 72

B. 2, 16, 5, 28, 12, 60, 32, 72

C. 2, 12, 16, 5, 28, 32, 72, 60

D. 5, 2, 12, 28, 16, 32, 72, 60



考点2 内部排序总结

【2015】已知小根堆为8, 15, 10, 21, 34, 16, 12, 删除关键字8之后需重建堆, 在此过程中, 关键字之间的比较次数是

- A. 1
- B. 2
- C. 3
- D. 4



考点2 内部排序总结

【2015】已知小根堆为8，15，10，21，34，16，12，删除关键字8之后需重建堆，在此过程中，关键字之间的比较次数是

- A. 1
- B. 2
- C. 3**
- D. 4



考点2 内部排序总结

【2021】将关键字6，9，1，5，8，4，7依次插入初始为空的大根堆H，得到的H是

- A. 9，8，7，6，5，4，1
- B. 9，8，7，5，6，1，4
- C. 9，8，7，5，6，4，1
- D. 9，6，7，5，8，4，1



考点2 内部排序总结

【2021】将关键字6，9，1，5，8，4，7依次插入初始为空的大根堆H，得到的H是

A. 9，8，7，6，5，4，1

B. 9，8，7，5，6，1，4

C. 9，8，7，5，6，4，1

D. 9，6，7，5，8，4，1



考点2 内部排序总结

【2021】设数组 $S[] = \{93, 946, 372, 9, 146, 151, 301, 485, 236, 327, 43, 892\}$ ，采用最低位优先（LSD）基数排序将 S 排列成升序序列。第一趟分配、收集后，元素372之前、之后紧邻的元素分别是

- A. 43, 892
- B. 236, 301
- C. 301, 892
- D. 485, 301



考点2 内部排序总结

【2021】设数组 $S[] = \{93, 946, 372, 9, 146, 151, 301, 485, 236, 327, 43, 892\}$ ，采用最低位优先（LSD）基数排序将 S 排列成升序序列。第一趟分配、收集后，元素372之前、之后紧邻的元素分别是

- A. 43, 892
- B. 236, 301
- C. 301, 892**
- D. 485, 301

考点3 外部排序

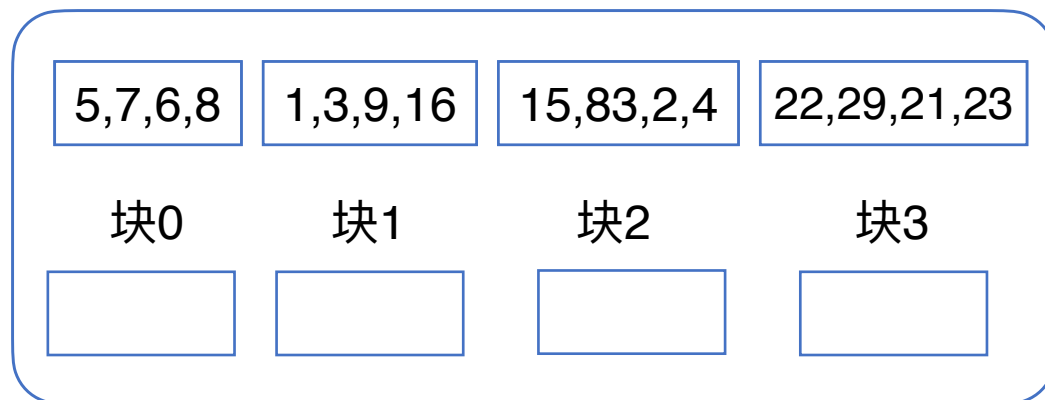
①思想

①待排序的记录数目很大时，无法一次性调入内存，整个排序过程就必须借用外存分批调入内存才能完成

②外部排序分为两步：

①通过内部排序获得初始归并段：按照可用内存的大小分批次将外存上的文件分成若干子文件，将其依次读入内存并利用有效的内部排序方法对它们进行排序得到**归并段**（有序的子文件）

②将初始归并段进行归并：对**归并段**进行逐趟归并，使其逐渐由小增大，直到获得整个有序文件



块4 块5 块6 块7

拥有8个块的磁盘



只能装下12个数字的
小的可怜的内存



能比较大小的cpu

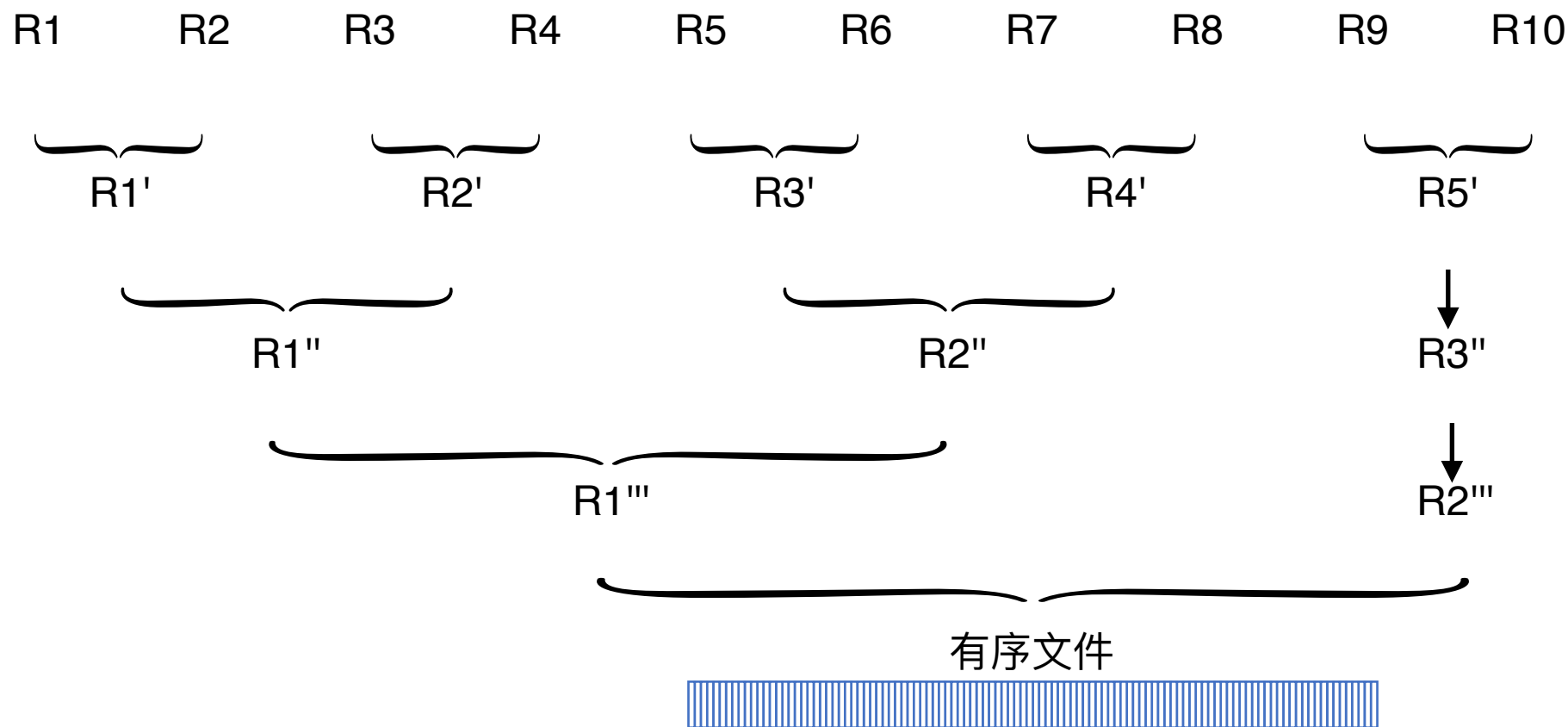


考点3 外部排序

②IO次数分析

选择题考频：2/16，分值：2

大题考频：1/16，分值：10



对外存信息的读写是以磁盘块为单位的，假设初始归并段共占用 x 个磁盘块
那么生成初始归并段需要 $\mathbf{【x*2】}$ 次读写（IO），二路归并需要 $\mathbf{【x*2*趟数】}$ 次读写



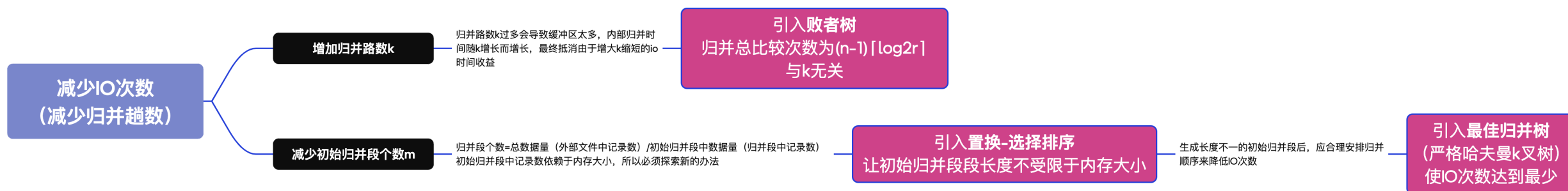
考点3 外部排序

③外部排序的时间优化

外部排序总时间=内部排序时间+外存信息读/写时间+内部归并时间

由于外存的读写时间远大于内部排序、内部归并的时间，所以对外部排序的时间优化目标就是减少io次数

一般情况下，对m个初始归并段进行k路平衡归并时，归并趟数 $s = \lceil \log_k m \rceil$

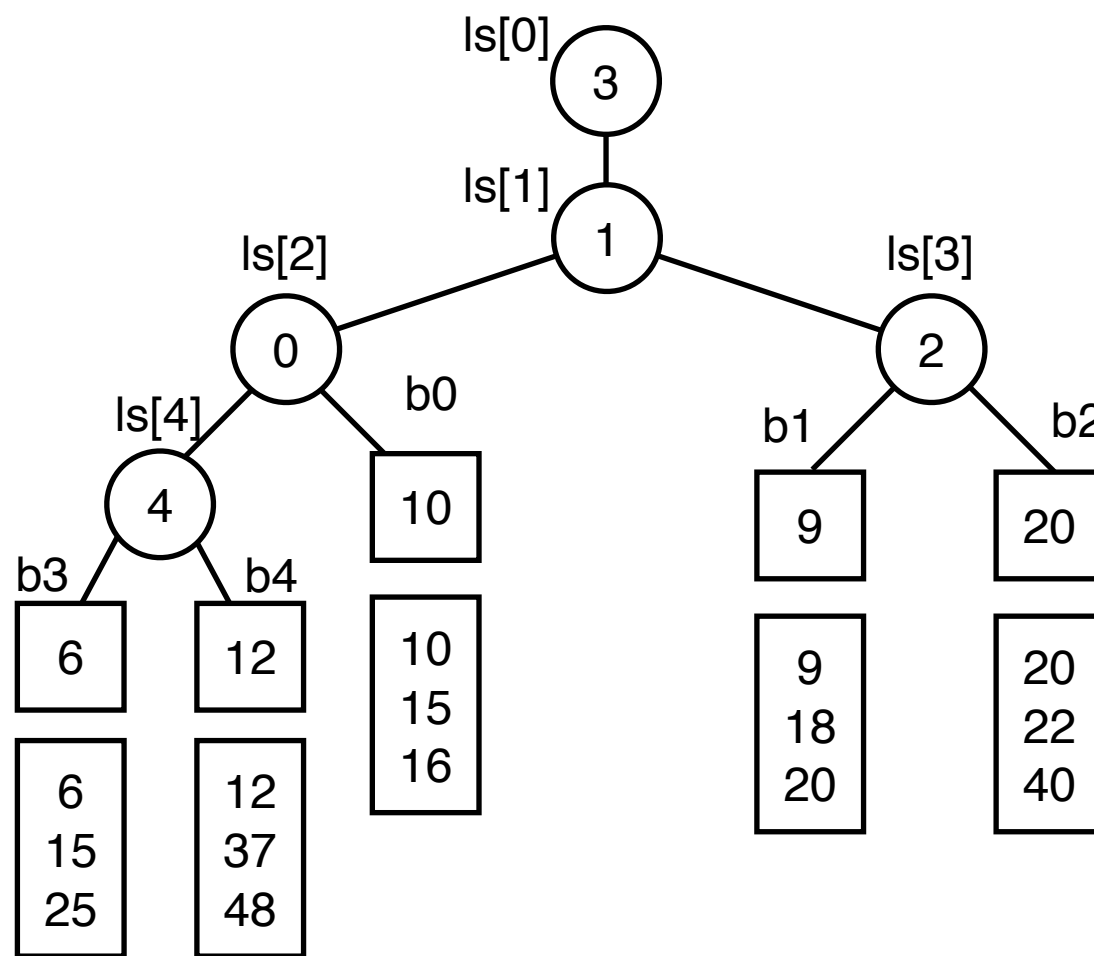


考点3 外部排序

③外部排序的时间优化

败者树

胜者继续比较，败者留下姓名（段号）
记录冠军的结点保存最小关键字所在的初始归并段段号



考点3 外部排序

③外部排序的时间优化

置换-选择排序

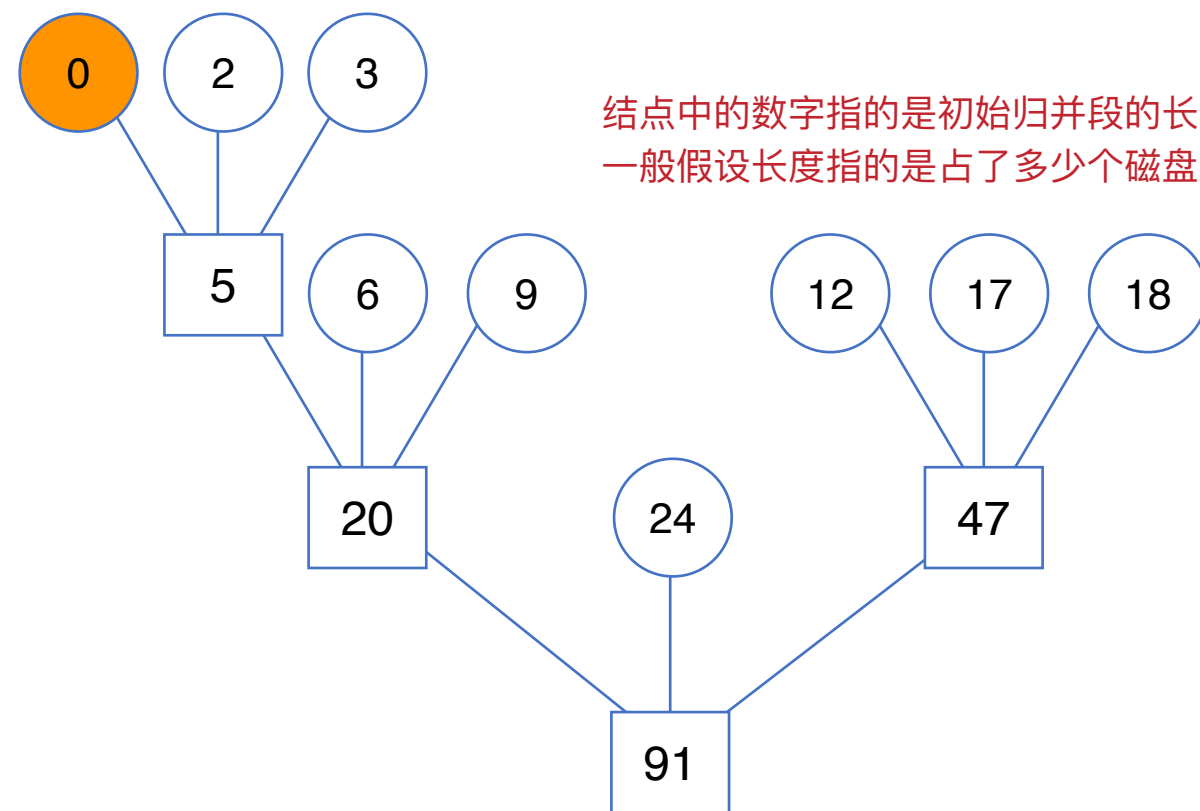
从待排序文件中输入关键字到工作区，从中选出最小值并记为MIN
将MIN关键字输出到输出文件
若待排序文件不空，则输入下一个关键字到工作区
从工作区中所有比MIN大的关键字中再选出最小值，作为新的MIN
直到工作区中选不出新的MIN，由此得到第一个初始归并段
重复上述过程，直到工作区空，即可得到全部初始归并段

输出文件FO	工作区WA（大小为4）	输入文件FI
		51943792146315994856 2360311743890166100

考点3 外部排序

③外部排序的时间优化

最佳归并树



对严格k叉树有 $n_0 = (k-1)n_k + 1$

$$\text{故 } n_k = \frac{n_0 - 1}{k - 1}$$

若 $(n_0 - 1) \% (k - 1) = 0$ ，则不需要添加虚段

若 $(n_0 - 1) \% (k - 1) = u \neq 0$ ，则需再加 $k - u - 1$ 个虚段

所需读写次数总共为

$$(0+2+3)*2+(5+6+9)*2+(12+17+18)*2+(20+24+47)*2=326$$

考点3 外部排序

【2019】设外存上有120个初始归并段，进行12路归并时，为实现最佳归并，需要补充的虚段个数是

- A. 1
- B. 2
- C. 3
- D. 4

考点3 外部排序

【2019】设外存上有120个初始归并段，进行12路归并时，为实现最佳归并，需要补充的虚段个数是

- A. 1
- B. 2**
- C. 3
- D. 4



考点3 外部排序

【2023】对含有 n ($n>0$) 个记录的文件进行外部排序，采用置换-选择排序生成初始归并段时需要使用一个工作区，工作区中能保存 m 个记录。请回答：

- 1) 若文件中含有19个记录，其关键字依次是51, 94, 37, 92, 14, 63, 15, 99, 48, 56, 23, 60, 31, 17, 43, 8, 90, 166, 100，则当 $m=4$ 时，可生成几个初始归并段？各是什么？
- 2) 对任意的 m ($n \gg m > 0$)，生成的第一个初始归并段的长度最大值和最小值分别是多少？



祝你今天特别特别开心^•̀-^